

UNIVERSITY OF COLOMBO, DEPARTMENT OF
PHYSICS

PH 3032 - Embedded Systems Laboratory
Final Project Report

3D LED Cube
Space Invasion Game

Name: Dinidu Kiriella

Index: s16102

Contents

1. Abstract	3
2. Introduction.....	4
3. Methodology	5
4. Result and Analysis	9
5. Discussion and Conclusion.....	10
6. Reference and Appendix	11

1. Abstract

This report provides a comprehensive overview of the design, construction, and programming of a 5x5x5 LED cube for an interactive space invasion game. The primary goal of the project was to control 125 individual LEDs using the ATmega328p microcontroller, which has limited I/O pins, and to creatively manipulate them for gameplay. The core gameplay involves the player controlling a 3x5 LED grid, referred to as the "ship," using two push buttons to move it up and down. Barriers, represented by LEDs lighting up randomly on the opposite face of the cube, move toward the ship, and the player must dodge these obstacles to continue the game. The objective is to survive as long as possible without the ship being destroyed by the incoming barriers. This project required careful integration of hardware and software. Key components included 3mm LEDs, resistors, transistors, 74HC238 decoders, and hex inverters. One of the major challenges was addressing each LED individually, which was solved using a multiplexer approach. The control system was designed such that each of the 25 columns of the cube could be individually addressed, with four 3-to-8 decoders handling 24 columns and a single microcontroller pin managing the last column. Additionally, the cube's row selection was managed by grounding the cathode terminals through programmable pins. Managing the power supply for all 125 LEDs was another challenge. Each LED required 1.9V to 2.1V and 20mA for optimal brightness, which demanded an efficient method for switching power to individual LEDs. This issue was resolved using transistors as switches, ensuring that only the necessary LEDs were powered without drawing excess current. The project was divided into three main components: the construction of the LED cube, the development of the addressing system, and the design of the control software. Building the physical structure required precision, as each of the five 5x5 horizontal planes needed to be perfectly aligned and soldered together. A custom LED driver circuit was also developed to interface the microcontroller with the cube, ensuring that each LED responded correctly to input signals. This project demonstrated the feasibility of using a 3D LED cube for interactive applications and provided valuable insights into the challenges and solutions involved in building complex embedded systems.

2. Introduction

An LED cube is a three-dimensional array of light-emitting diodes (LEDs) arranged in a cubic structure. Each LED in the cube can be individually controlled to turn on or off, allowing the display of patterns, animations, or even simple games in a three-dimensional space. The size of the cube can vary, with common examples being 4x4x4, 5x5x5, or even larger.

For this project, a 5x5x5 LED cube was used, and for this project simplified version space invader game was used. Using two pushbutton 5x3 grids is controlled and can be used as a ship in the space invasion game. Barriers come toward the ship from the opposite face of the cube. Barriers are created at random places on the opposite side of the cube from the ship.

3D LED cubes can display 3D images and animations, creating stunning visual effects using their dimensions. They can also display different content on each side, making them interactive and engaging therefore LED cubes make a great choice for displaying a game and making more interaction for the player. Also, 3D LED cubes are durable and have low maintenance costs, making them a cost-effective solution in the long run. Using the ATmega328p microcontroller, all the software and hardware are coordinated.

Although there were a few challenges like addressing an LED from among 125 LEDs, supplying the controlled voltage to all 125 LEDs, and building the Cube structure, they were overcome by using a few techniques. The multiplexer method is used to solve the challenge of addressing each LED from the microcontroller. Using transistors as a power switch overcomes the problem of the power supply of LEDs.

3. Methodology

Hardware Description

First Making the 5x5x5 cube structure was challenging without proper measurement. Therefore, the LED cube started to be built after building the 5 horizontal 5x5 plane individually using the 5x5 holes frame and soldering all the negative terminals of the LED's onto the wire and leaving the positive terminal hanging. Then carefully solder the 5 horizontal planes together by soldering the LED anode terminals of each plane on one another plane.

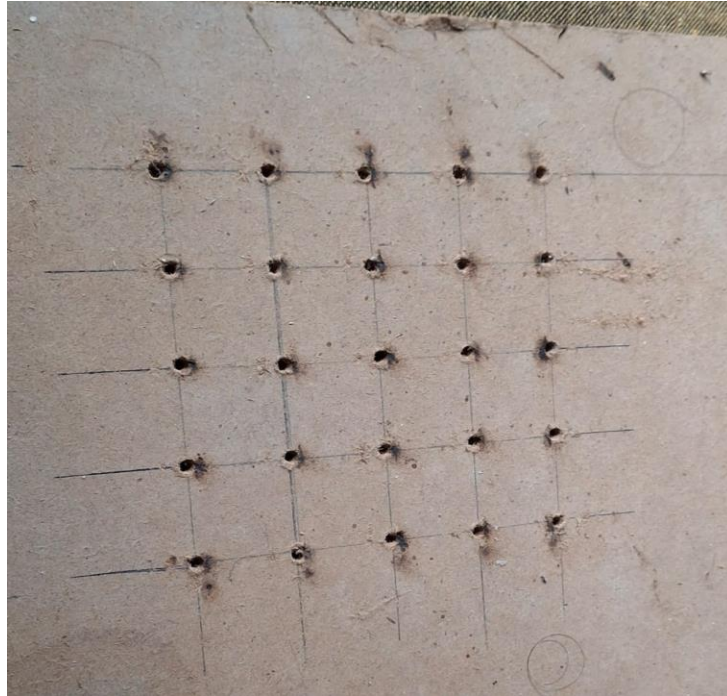


Figure 1: Wooden frame to create 5x5 LED planes

In addition to building the LED cube itself, a Custom LED driver circuit was designed and built. The circuit took the inputs from a few microcontroller pins and decoded them into something that could control the cube. Using this circuit, only 11 microcontroller pins decided which LED must light up from 125 LEDs in the cube. For that multiplexer, the method was used and using four 3 to 8 decoders 24 columns of the cube are addressed and c5 pins control the 25th column and the matter of which row needs is chosen by the grounded cathode planes from their row assign pins making them high voltages to low voltages using the program. One of the decoders chooses which decoder is needed to address and the rest of the three decoders address 24 columns with each decoder addressing 8 columns. Before the decoder output is connected to the LED cube their outputs go through the hex inverters and another circuit part.

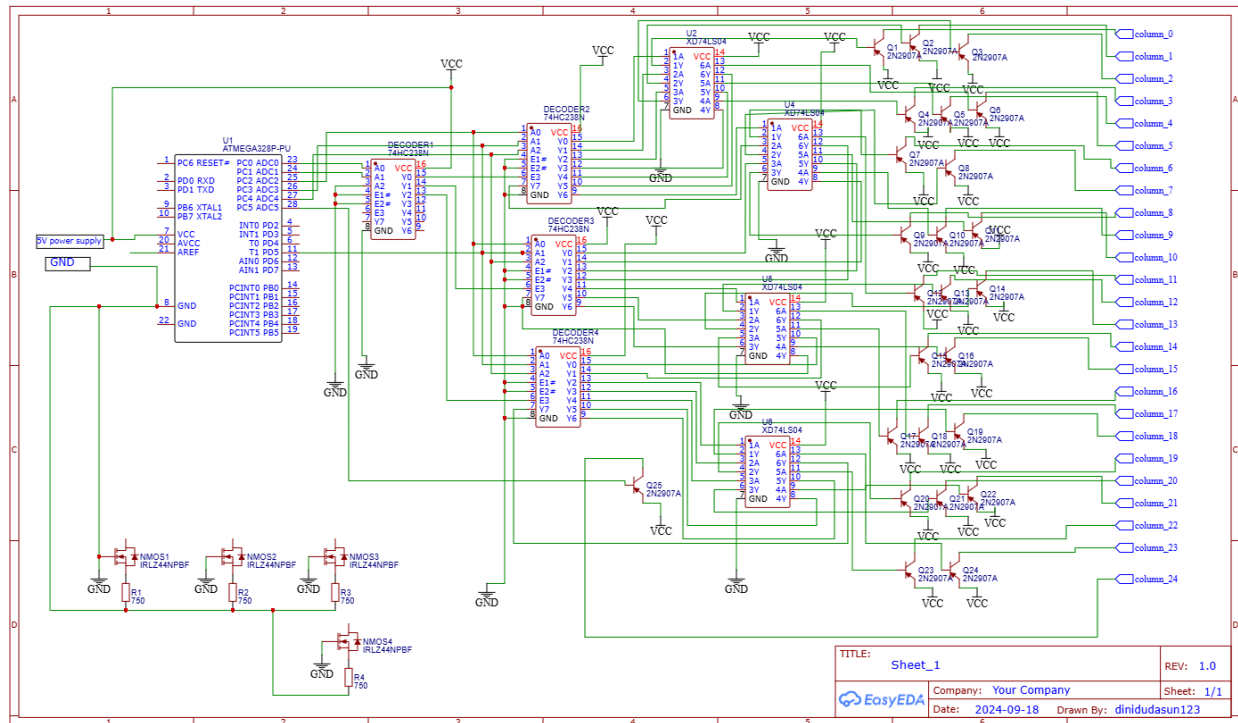


Figure 2: Schematic diagram of full circuit of LED drive circuit

Another challenge of this project is the power supply to each LED without affecting another LEDs. Therefore transistors are used in this scenario as power switches to power each LED when the transistor gets the signal from the inverters. After finalizing the circuit which gave reasonable LED brightness and controls PCB was designed. PCB was designed as two separate parts because of the complexity of the circuit.

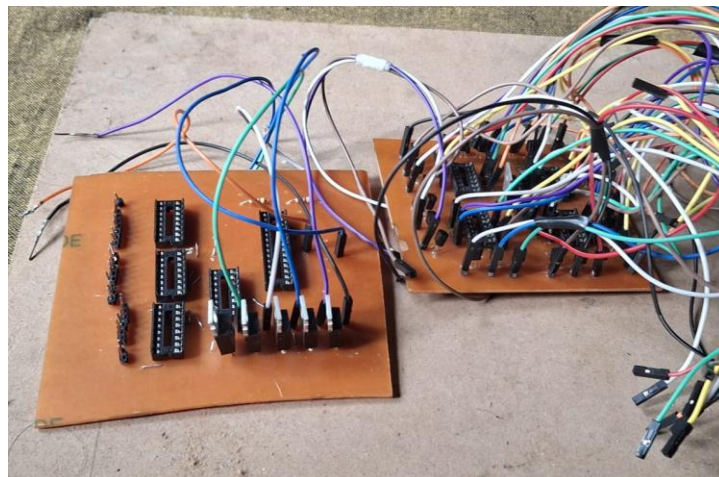


Figure 3: Two PCB circuits, One PCB includes a microcontroller, 4 Decoder and N-MOSFETs (On the Left side), The Other PCB includes hex inverters and a jumper wire connection to connect the LED cube (On the Right side)

Software Description

The control software for the LED cube has a very simple structure. The initialization part was coded first. The PORTC and PORTD pins, which are connected to the cube through the LED Drive circuit, are initialized as the Outputs. Initially, PORTC pins used through decoders gave low voltage, and the PORTD pins, which are controlled layers of the cube, gave high voltage. C0 and C1 pins were used to determine which decoder address through decoder 1. C2, C3 and C4 pins were used to determine the which output need to call from 0 - 7 of each decoder.

0 1 2 3 4

5 6 7 8 9

10 11 12 13 14

15 16 17 18 19

20 21 22 23 24

For easier identification 25 columns were labeled as above. 0 - 7 columns are controlled by decoder 2, 8 - 15 columns are controlled by decoder 3, 16 - 23 columns are controlled by decoder 4 and column 24 is directly controlled by the C5 pin. If we needed to light up the bulb in column 8 layer 2, then choose the address of the decoder through decoder 1 making C0 pin - 0, C1 pin - 1. After that choose the assigned output to column 8 according to decoder 2 outputs.

Table 1: Example Pin Configuration

Example column	C0 pin (Logic)	C1 pin (Logic)	C2 pin (Logic)	C3 pin (Logic)	C4 pin (Logic)
10	1	0	0	1	0
19	0	1	0	0	1

Even though one LED or LEDs belonging to the same layer could light up this way it does not supply the visualization of a LED cube because according to setup one layer can light up at a time. Therefore we light up every layer with a delay while making a visual effect on the human eye that every layer lights up at the same time. For two buttons use the debounce function for the ship to go up and down. To create Space Invader game in a LED cube first light up layers 2,3,4 in the 0, 1, 2, 3 and 4 columns as a ship. Using the built-in "rand" function light up the random LED in 20, 21, 22, 23 and 24 columns including all the layers. And using other functions to make sure when the ship is hit by a random bulb that coming toward the ship or misses the ship completes the game in the LED cube.

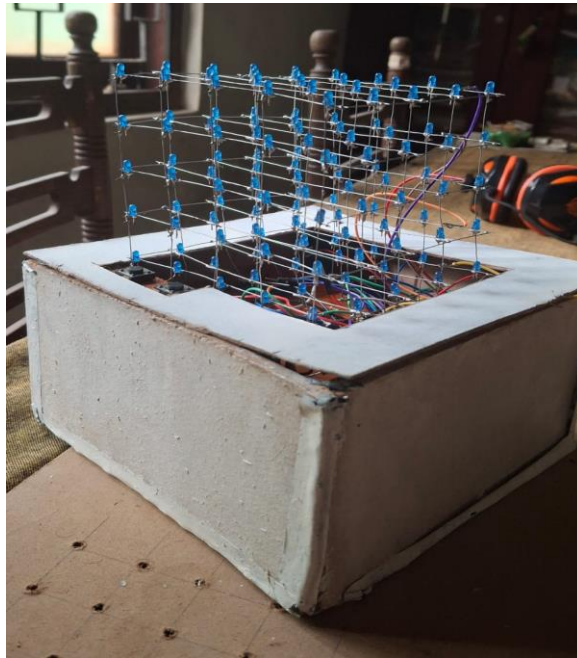


Figure 4: LED Cube with the Wooden box

4. Result and Analysis

The cube was able to light up relevant LEDs to ship and barriers (random LEDs) of the Space Invader game while lighting up the 5 columns per time. Also, the LED cube responded to buttons quickly and the ship part of the game responded to buttons. However it was challenging to observe the full working cube without the PCB as it was a complex circuit with 8 ICs, 25 transistors and 5 N-MOSs. After assembling the full circuit with 2 PCB circuits (because of the complexity of the circuit and PCB circuit, 2 pieces of PCB were used). But after assembling the PCBs and Cube and buttons, the LED cube didn't work as supposed to work. Some random LEDs in the cube start to light up with dim light.

Our final product was both usable and safe. The cube is powered by the 5V and 1A adapter and all the necessary components are contained within the base in a wooden box so the device is both portable and safe. The cube also produces minimal interference with other projects since the only thing that we really 'emit' is blue light. After checking the PCBs there were a few shorts and therefore after building another 2 pieces of PCBs there were soldering problems and didn't give the overall supposed results in the given project time.

However, the LED's were as bright as had hoped and in a well-lit room or on a very sunny day it can be difficult to see what the cube is trying to display. This problem could be fixed in the future by making the PCBs in bigger copper boards with more clearance of the pads and lines in the circuit and better soldering work.

5. Discussion and conclusion

The 3D LED cube project demonstrated the design and control of a 5x5x5 LED matrix as a playable space invasion game. The implementation involved hardware challenges such as addressing individual LEDs among 125 units and managing the power distribution across all LEDs. While the design was functional, several lessons were learned, and improvements were suggested for future versions.

The project encountered significant challenges in constructing the physical structure of the cube and designing the custom LED driver circuit. The choice of using transistors as power switches effectively addressed the issue of providing sufficient current to the LEDs while maintaining control over individual units. However, assembling the PCBs was a challenging process. With 8 ICs, 25 transistors, and two pushbuttons, ensuring proper connections and avoiding shorts required precise soldering work. Issues arose when the initial PCB assembly resulted in random LEDs lighting up with dim brightness, indicating the presence of shorts or incorrect wiring.

The decision to split the circuit into two PCBs helped reduce the complexity, but the additional PCB design introduced new challenges in terms of alignment and debugging. Ultimately, the PCBs had to be rebuilt to address soldering problems. This experience highlights the importance of careful PCB design, including sufficient pad clearances and optimized component placement, to reduce assembly errors.

The LED cube project provided valuable experience in designing both the hardware and software components of a complex embedded system. It demonstrated the feasibility of creating an interactive 3D game using a 5x5x5 LED matrix, controlled by an ATmega328p microcontroller. While the final product was functional and safe, technical challenges such as PCB shorts and soldering issues prevented the cube from achieving its supposed performance within the project's timeframe. The project was a valuable learning experience, demonstrating both the complexity and potential of 3D LED cubes in creating interactive displays. Despite the challenges faced, the project successfully achieved many of its objectives and laid the foundation for further development.

6. Reference and Appendix

Reference:-

(No date a) *3D led cube - ECE (UCF.EDU) - University of Central Florida*. Available at: <https://www.ece.ucf.edu/seniordesign/fa2013sp2014/g15/docs/sd1.pdf> (Accessed: 15 August 2024).

(No date b) *8x8x8 animated led Cube Project Review | Jameco Electronics Video*. Available at: <https://www.jameco.com/Jameco/workshop/Video/8x8x8-Animated-LED-Cube.html> (Accessed: 18 August 2024).

5x5x5 led Cube - orientation independent 3D display (no date) *3D LED Cube*. Available at: https://people.ece.cornell.edu/land/courses/ece4760/FinalProjects/s2008/pae26_rwc28/pae26_rwc28/index.html (Accessed: 10 August 2024).

Chr and Instructables (2023) *Led cube 8x8x8, Instructables*. Available at: <https://www.instructables.com/Led-Cube-8x8x8/> (Accessed: 20 August 2024).

Gothwal, P. and Singh Shekhawa, S. (2018) 'Three dimensional cube', *International Journal of Engineering & Technology*, 7(3.30), p. 90. doi:10.14419/ijet.v7i3.30.18207.

Appendix:-

Table 2: List of Components

Part	Quantity
ATmega328p Microcontroller	1
CD74HCT238 3-8 Decoders	4
74LS04 Hex Inverter	4
3mm LED	125
2n2907a Transistor	25
IRFZ44N transistors	5
Pushbuttons	2
5V and 1A Power Adapter	11
14-pin IC base	4
28-pin IC base	1
16-pin IC base	4
Various Resistors, and pin headers and wires	-

Led_cube.h

```
#ifndef LED_CUBE_H
#define LED_CUBE_H

#include <avr/io.h>
#include <util/delay.h>
#include <stdlib.h> // For random number generation

#define F_CPU 1000000UL

// Column control pins
#define C0_PIN PC0
#define C1_PIN PC1
#define C2_PIN PC2
#define C3_PIN PC3
#define C4_PIN PC4
#define C5_PIN PC5

// Layer control pins
#define LAYER1_PIN PD0
#define LAYER2_PIN PD1
#define LAYER3_PIN PD2
#define LAYER4_PIN PD3
#define LAYER5_PIN PD4

// Button pins
#define BUTTON_UP_PIN PB6
#define BUTTON_DOWN_PIN PB7

void initialization(void);
```

```

void select_columns(uint8_t col);

void select_layers(uint8_t start_layer);

void display_layers_with_column(uint8_t start_layer, uint8_t col);

uint8_t debounce_button(uint8_t buttonPin);

void light_random_leds(uint8_t layer);


void initialization(void) {
    // column control pins as outputs
    DDRC |= (1 << C0_PIN) | (1 << C1_PIN) | (1 << C2_PIN) | (1 << C3_PIN) | (1 << C4_PIN);
    PORTC &= ~((1 << C0_PIN) | (1 << C1_PIN) | (1 << C2_PIN) | (1 << C3_PIN) | (1 << C4_PIN) | (1 <<
C5_PIN));

    // layer control pins as outputs
    DDRD |= (1 << LAYER1_PIN) | (1 << LAYER2_PIN) | (1 << LAYER3_PIN) | (1 << LAYER4_PIN) | (1 <<
LAYER5_PIN);
    PORTD &= ~((1 << LAYER1_PIN) | (1 << LAYER2_PIN) | (1 << LAYER3_PIN) | (1 << LAYER4_PIN) | (1
<< LAYER5_PIN));

    // button pins
    DDRB &= ~((1 << BUTTON_UP_PIN) | (1 << BUTTON_DOWN_PIN));
    PORTB |= (1 << BUTTON_UP_PIN) | (1 << BUTTON_DOWN_PIN); // Enable pull-up resistors
}


void select_layers(uint8_t start_layer) {
    // Clear
    PORTD &= ~((1 << LAYER1_PIN) | (1 << LAYER2_PIN) | (1 << LAYER3_PIN) | (1 << LAYER4_PIN) | (1
<< LAYER5_PIN));

    // Light up 3 consecutive layers starting
    switch (start_layer) {
        case 1:
            PORTD |= (1 << LAYER1_PIN) | (1 << LAYER2_PIN) | (1 << LAYER3_PIN);

```

```

        break;
    case 2:
        PORTD |= (1 << LAYER2_PIN) | (1 << LAYER3_PIN) | (1 << LAYER4_PIN);
        break;
    case 3:
        PORTD |= (1 << LAYER3_PIN) | (1 << LAYER4_PIN) | (1 << LAYER5_PIN);
        break;
    case 4:
        PORTD |= (1 << LAYER4_PIN) | (1 << LAYER5_PIN) | (1 << LAYER1_PIN); // Wrap around
        break;
    case 5:
        PORTD |= (1 << LAYER5_PIN) | (1 << LAYER1_PIN) | (1 << LAYER2_PIN); // Wrap around
        break;
}
}

```

```

void select_columns(uint8_t col) {
    // Clear
    PORTC &= ~((1 << C2_PIN) | (1 << C3_PIN) | (1 << C4_PIN));

    switch (col) {
        case 0:
            break; // All pins are already cleared
        case 1:
            PORTC |= (1 << C2_PIN);
            break;
        case 2:
            PORTC |= (1 << C3_PIN);
            break;
        case 3:

```

```

        PORTC |= (1 << C2_PIN) | (1 << C3_PIN);

        break;

    case 4:

        PORTC |= (1 << C4_PIN);

        break;

    case 21:

        PORTC = (0 << C0_PIN) | (1 << C1_PIN) | (0 << C2_PIN) | (0 << C3_PIN) | (1 << C4_PIN);

        break;

    case 22:

        PORTC = (0 << C0_PIN) | (1 << C1_PIN) | (1 << C2_PIN) | (0 << C3_PIN) | (1 << C4_PIN);

        break;

    case 23:

        PORTC = (0 << C0_PIN) | (1 << C1_PIN) | (0 << C2_PIN) | (1 << C3_PIN) | (1 << C4_PIN);

        break;

    case 24:

        PORTC = (0 << C0_PIN) | (1 << C1_PIN) | (1 << C2_PIN) | (1 << C3_PIN) | (1 << C4_PIN);

        break;

    case 25:

        PORTC = (1 << C5_PIN); // directly connected to C5

        break;

    }

}

void display_layers_with_column(uint8_t start_layer, uint8_t col) {

    select_layers(start_layer);

    select_columns(col);

}

uint8_t debounce_button(uint8_t buttonPin) {

    if (!(PINB & (1 << buttonPin))) {

```

```

        _delay_ms(50);
        if (!(PINB & (1 << buttonPin))) { // Check again after delay
            return 1; // button is pressed
        }
    }
    return 0;
}

void light_random_leds(uint8_t layer) {
    for (uint8_t i = 0; i < 5; i++) {
        // generate random column number between 1 and 5
        uint8_t random_col = (rand() % 5) + 21;
        display_layers_with_column(layer, random_col);
        _delay_ms(50);
    }
}

void turn_off_led(uint8_t layer, uint8_t column) {
    select_layers(layer);
    select_columns(column);
    _delay_ms(50);
    // clear the layer and column after the delay
    PORTC &= ~((1 << C0_PIN) | (1 << C1_PIN) | (1 << C2_PIN) | (1 << C3_PIN) | (1 << C4_PIN));
    PORTD &= ~((1 << LAYER1_PIN) | (1 << LAYER2_PIN) | (1 << LAYER3_PIN) | (1 << LAYER4_PIN) | (1
<< LAYER5_PIN));
}

```

#endif

main.c

```
#include "led_cube.h"
```

```
#include <stdlib.h>
```

```
int main(void) {
```

```
    uint8_t current_layer = 2; // Start from layer 2
```

```
    initialization();
```

```
    uint8_t random_layer = rand() % 5 + 1;
```

```
    uint8_t random_column = (rand() % 5) + 21;
```

```
    while (1) {
```

```
        // Up button
```

```
        if (debounce_button(BUTTON_UP_PIN)) {
```

```
            current_layer--;
```

```
            if (current_layer < 1) current_layer = 5;
```

```
        }
```

```
        // Down button
```

```
        if (debounce_button(BUTTON_DOWN_PIN)) {
```

```
            current_layer++;
```

```
            if (current_layer > 5) current_layer = 1;
```

```
        }
```

```
        // move the random LED
```

```
        for (uint8_t layer = 5; layer >= 1; layer--) {
```

```
            if (layer == current_layer && random_column == current_layer) {
```

```
                // Collision
```

```
                turn_off_led(current_layer, random_column); //turn off the hit LED
```

```
            } else {
```

```
                display_layers_with_column(layer, random_column);
```

```
                _delay_ms(50);
```

```

    }
}

// generate new random LED
if (random_layer == 1) {
    random_layer = rand() % 5 + 1;
    random_column = (rand() % 5) + 1;
}

// ship layer
for (uint8_t col = 1; col <= 5; col++) {
    display_layers_with_column(current_layer, col);
    _delay_ms(25);
}
}

return 0;
}

```